

MANIPAL
ACADEMY of HIGHER EDUCATION
(Institution of Eminence Deemed to be University)

MANIPAL SCHOOL OF INFORMATION SCIENCES
(A Constituent unit of MAHE, Manipal)

Linux OS and Scripting – 6_Lab_D2_AWK

Reg. Number	Name	Branch
261100690006	MOHAMMED HARIS	Cybersecurity



MANIPAL SCHOOL OF INFORMATION SCIENCES
MANIPAL
(A constituent unit of MAHE, Manipal)

Write an AWK program for each question. Ignore the header row where appropriate.

1. Print the source IP and event from every record.

```
BEGIN {
    FS = ","
}
NR == 1 { next }
{
    print $2, $3
}
```

2. Display the time, source IP, and attempts in aligned columns using printf.

```
BEGIN {
    FS = ","
}
NR == 1 { next }
{
    printf "%-15s %-20s %s\n", $1, $2, $5
}
```

3. Store the source IP and event in variables, then print them in a sentence.

```
BEGIN {
    FS = ","
}
NR == 1 { next }
{
    ip = $2
    event = $3

    print "IP is", ip, "and event is", event
}
```

4. Calculate the processing time for each record if one attempt takes 2 seconds.

```
BEGIN {
    FS = ","
}
NR == 1 { next }
{
    time = $5 * 2

    print $2, time, "seconds"
}
```

5. Calculate the total number of attempts in the file.

```
BEGIN {
    FS = ","
    total = 0
}
NR == 1 { next }
{
    total = total + $5
}
```

```
END {
    print "Total attempts:", total
}
```

6. Calculate the average number of attempts per record.

```
BEGIN {
    FS = ","
    total = 0
    count = 0
}
NR == 1 { next }
{
    total = total + $5
    count++
}
END {
    print "Average:", total / count
}
```

7. Count the number of Login_Failed records.

```
BEGIN {
    FS = ","
    count = 0
}
NR == 1 { next }
{
    if ($3 == "Login_Failed") {
        count++
    }
}
END {
    print "Login Failed:", count
}
```

8. Start with a review quota of 10 and decrease it by one for every HIGH or CRITICAL record. Print the remaining quota.

```
BEGIN {
    FS = ","
    quota = 10
}
NR == 1 { next }
{
    if ($4 == "HIGH" || $4 == "CRITICAL") {
        quota--
    }
}
END {
    print "Remaining quota:", quota
}
```

9. Replace Port_Scan with Network_Scan in the event field and print every record in CSV format.

```
BEGIN {
    FS = ","
    OFS = ","
}
NR == 1 {
    print $0
    next
}
{
    if ($3 == "Port_Scan") {
        $3 = "Network_Scan"
    }

    print $1, $2, $3, $4, $5
}
```

10. Add one to the attempts field of each Login_Failed record and print every record in CSV format.

```
BEGIN {
    FS = ","
    OFS = ","
}
NR == 1 {
    print $0
    next
}
{
    if ($3 == "Login_Failed") {
        $5 = $5 + 1
    }

    print $1, $2, $3, $4, $5
}
```

11. Print the source IP and event when severity is HIGH and attempts are greater than 5.

```
BEGIN {
    FS = ","
}
NR == 1 { next }
{
    if ($4 == "HIGH" && $5 > 5) {
        print $2, $3
    }
}
```

12. Print the source IP and event when severity is HIGH or CRITICAL.

```
BEGIN {
    FS = ","
```

```
}
NR == 1 { next }
{
    if ($4 == "HIGH" || $4 == "CRITICAL") {
        print $2, $3
    }
}
```

13. Use the conditional operator to label each record Repeated if attempts exceed 1; otherwise label it Single.

```
BEGIN {
    FS = ","
}
NR == 1 { next }
{
    result = ($5 > 1) ? "Repeated" : "Single"

    print $2, result
}
```

14. Use the conditional operator to label each record Investigate if attempts are at least 5; otherwise label it Routine.

```
BEGIN {
    FS = ","
}
NR == 1 { next }
{
    result = ($5 >= 5) ? "Investigate" : "Routine"

    print $2, result
}
```

15. Use if–else to print Review required for HIGH or CRITICAL records and Routine review for all other records.

```
BEGIN {
    FS = ","
}
NR == 1 { next }
{
    if ($4 == "HIGH" || $4 == "CRITICAL") {
        print "Review required"
    }
    else {
        print "Routine review"
    }
}
```

16. Use if–else if–else to classify attempts as High activity (10 or more), Moderate activity (5–9), or Low activity (fewer than 5).

```
BEGIN {
    FS = ","
}
NR == 1 { next }
{
    if ($5 >= 10) {
        print "High activity"
    }
    else if ($5 >= 5) {
        print "Moderate activity"
    }
    else {
        print "Low activity"
    }
}
```

17. Use a for loop to print Check 1 through Check 5.

```
BEGIN {
    for (i = 1; i <= 5; i++) {
        print "Check", i
    }
}
```

18. Use a while loop to count down from 10 to 1.

```
BEGIN {
    i = 10

    while (i >= 1) {
        print i
        i--
    }
}
```

19. Use a do–while loop to count down from 10 to 1.

```
BEGIN {
    i = 10

    do {
        print i
        i--
    } while (i >= 1)
}
```

20. Print each record's source IP, event, attempts, and activity classification in aligned columns. At the end, print the total records and total attempts.

```
BEGIN {
    FS = ","
    total = 0
```

```
    count = 0

    printf "%-18s %-18s %-10s %-15s\n", "IP", "Event", "Attempts", "Activity"
}
NR == 1 { next }
{
    if ($5 >= 10) {
        activity = "High"
    }
    else if ($5 >= 5) {
        activity = "Moderate"
    }
    else {
        activity = "Low"
    }

    printf "%-18s %-18s %-10s %-15s\n", $2, $3, $5, activity

    total = total + $5
    count++
}
END {
    print "Total records:", count
    print "Total attempts:", total
}
```

Login monitoring: A security analyst considers five or more failed attempts suspicious. Write an AWK script that prints the source IP and attempts for matching Login_Failed entries. At the end, print how many suspicious entries were found.

```
BEGIN {
    FS = ","
    count = 0
}
NR == 1 { next }
{
    if ($3 == "Login_Failed" && $5 >= 5) {
        print $2, $5
        count++
    }
}
END {
    print "Suspicious entries:", count
}
```

Incident summary: Write an AWK script that counts the entries in each severity category and prints a summary in an END block.

```
BEGIN {
```

```
    FS = ","
}
NR == 1 { next }
{
    severity[$4]++
}
END {
    for (s in severity) {
        print s, severity[s]
    }
}
```

Processing estimate: Assume each attempt requires 2 seconds of analyst review. Calculate the review time for each entry and the total review time for the file. Use printf to present the results neatly.

```
BEGIN {
    FS = ","
    total = 0
}
NR == 1 { next }
{
    time = $5 * 2
    total = total + time

    printf "%-18s %-10s %-10s\n", $2, $5, time
}
END {
    print "Total review time:", total, "seconds"
}
```

Log normalization: A monitoring tool requires the event name Failed_Login instead of Login_Failed. Update the matching field and print all records as CSV. The source file should remain unchanged.

```
BEGIN {
    FS = ","
    OFS = ","
}
NR == 1 {
    print $0
    next
}
{
    if ($3 == "Login_Failed") {
        $3 = "Failed_Login"
    }

    print $1, $2, $3, $4, $5
}
```

Priority queue: Assign each entry a priority: Immediate for CRITICAL, Soon for HIGH, and Routine for all others. Print the time, source IP, event, and assigned priority.

```
BEGIN {
    FS = ","
}
NR == 1 { next }
{
    if ($4 == "CRITICAL") {
        priority = "Immediate"
    }
    else if ($4 == "HIGH") {
        priority = "Soon"
    }
    else {
        priority = "Routine"
    }

    print $1, $2, $3, priority
}
```